ALSTOM • IXL ENGINEERING SOLUTIONS

IXL Team Hours Tracker & Allocation System

Comprehensive Technical Architecture, System Internals, & Developer Manual

Version: 2.0.0 (Pure Technical) **Target:** Software Engineers & AI Agents **Production:** ixl-hours-tracker.web.app **Git:** main branch

---

> **Developer Maintenance Contract:** This document represents the authoritative single-source-of-truth technical blueprint for the entire application. Whenever any functionality, data structure, or endpoint is added, modified, or removed, this document MUST be updated accordingly.

1. Executive System Overview

The **Alstom IXL Team Hours Tracker** is a specialized high-performance Single Page Application (SPA) designed to track, schedule, and reconcile daily engineering hours for multi-regional railway signaling teams in the **Cairo** and **Istanbul** engineering offices.

Key architectural goals:

- **Double-Entry Accounting Reconciliation:** Project hours aggregated across the system strictly match individual team member hours at all times (Project Sum = Employee Sum).
- **Dual Office Calendars:** Cairo (Friday/Saturday weekend) vs. Istanbul (Saturday/Sunday weekend) automated compliance.
- **Deterministic Zero-Framework Architecture:** Vanilla ES6+, Semantic HTML5, and CSS3 with hardware-accelerated touch physics. Zero heavy build tools or framework overhead.
- **Zero-Emoji Standard:** Strict corporate constraint prohibiting Unicode emojis across UI text, code symbols, test suites, and commit logs.

```
[Client Web Browser]
   ├── Desktop (PC)    --> Excel-Style Grid, 2D Multi-Select, Keyboard Navigation
   └── Mobile Touch    --> Mobile Editor (Form/Cards), Agenda List, 1-Tap Starred Chips
        |
        ├── In-Memory State Machine: window.state
        ├── Local Storage & Cookie Fallback Cache (Synchronous optimistic updates)
        └── Network REST Sync: POST /api/sync (Atomic payload with SHA-256 validation)
             |
             ├── Node.js Server (server.js) OR PowerShell Server (server.ps1)
```

```
└── Google Firebase Hosting Edge CDN (Production)
```

2. Complete File Inventory & Code Map

File Path	Size	Architectural Role & Description
`index.html`	109 KB	Complete SPA markup shell: schedule table, mobile cards, popovers, modals, responsive pictures.
`app.js`	368 KB	Master client state engine (~8,800 lines): DOM event delegation, math reconciliation, state sync, identity persistence.
`styles.css`	141 KB	Design system (~6,100 lines): CSS variables, light/dark themes, responsive layout breakpoints, dual-axis touch physics.
`data.json`	20 KB	Master single-source-of-truth database: sections, employees, projects, allocations, admin PIN hash, audit log.
`data.backup.json`	14 KB	Cold-start fallback image used for automatic recovery by local servers.
`server.js`	3.8 KB	Node.js Express REST API server providing `GET /api/data` and `POST /api/sync`.
`server.ps1`	8.5 KB	Native Windows PowerShell HTTP listener for restricted enterprise systems lacking Node.js runtime.
`firebase.json`	675 B	Firebase Hosting configuration: route rewrites, CDN cache-control headers, deploy ignore rules.
`cloudflared.exe`	55.3 MB	Cloudflare Tunnel binary for instant public link generation via `Share-Public-Link.bat`.
`test_*.js & *.ps1`	~200 KB	12 automated test suites verifying 2,150+ unit, integration, and security assertions.

3. Master Data Models & State Architecture

3.1 The In-Memory State Object (`window.state`)

```
window.state = {
  sections: [],         // Office sections: Cairo, Istanbul
  employees: [],        // 26 Team members (active & deactivated)
  projects: [],         // 55 Projects, tenders, training & support
  allocations: {},      // Hash map: "empId_YYYY-MM-DD" -> { project, hours }
  auditHistory: [],     // Chronological security and edit events
  adminPasswordHash: "",// SHA-256 hash for Peter MOUNIR admin mode
  currentUserId: null,  // Active user profile ID ("emp_1", ...)
  isAdmin: false,       // Master admin privilege flag
  isViewOnly: false,    // Read-only URL parameter flag (?mode=view)
  currentYear: 2026,    // Active calendar year (starts Nov 2026)
  currentMonth: 11,     // Active calendar month (1-based: 11 = November)
  deviceMode: "pc",     // "pc" or "mobile"
  mobileViewMode: "form", // "form" (Editor) or "grid" (Viewer)
  selectedCells: new Set(), // Set of active composite keys in 2D multi-select
  activeKeyboardCell: null  // Composite key holding active keyboard focus
};
```

3.2 Composite Key Architecture

Allocations are keyed by composite strings formatted as `${employeeId}_${YYYY-MM-DD}` (e.g. `emp_22_2026-11-04`).

To prevent string splitting bugs when employee IDs contain underscores (such as `emp_22`), keys are parsed using `lastIndexOf('_')` :

```
const lastUnderscore = key.lastIndexOf('_');
const empId = key.substring(0, lastUnderscore);
const dateStr = key.substring(lastUnderscore + 1);
```

4. Dual Office Calendar & Weekend Rules

Office Section	Location	Official Weekend (Holidays)	Official Workdays
Applications - Cairo	Cairo	Friday (5), Saturday (6)	Sunday through Thursday
ILS - Istanbul	Istanbul	Saturday (6), Sunday (0)	Monday through Friday

Multi-Day Range vs. Single Weekend Overtime Rule

- **Multi-Day Range:** When filling a date range that spans a weekend, the chosen holiday option is applied:
  - `dayoff`: Weekends are set to `DAY OFF` with 0h. Workdays get the project.
  - `include`: Weekends are worked and filled with project and hours.
  - `skip`: Weekends are untouched.
- **Explicit Single Weekend Day:** If an engineer clicks a single weekend day (e.g. Friday in Cairo), the system permits direct overtime logging without forcing `DAY OFF`.

5. Desktop (PC) Interface Architecture

- **Excel-Style Grid Navigation:** Arrow keys move the focused cell across days and employees. `Enter` opens the popover. `Delete` clears allocations.
- **Pointer State Machine:**
  - Single Click: Stands on cell without opening popovers (cleans keyboard focus).
  - Double Click: Opens the Project Selection Popover.
  - 380ms Long Press: Enters 2D rectangular multi-cell selection.
  - 8px Jitter Deadband: Prevents micro-mouse tremors from accidentally entering drag mode.
- **Admin 2D Multi-Selection:** In Peter MOUNIR's Admin Mode, multi-cell drag expands across multiple employee rows and dates simultaneously.

5.1 Employee Monthly Contributions Modal (#empContribModal)

Triggered by double-clicking any employee row on the schedule grid (Desktop) or via member contributions inspection (Mobile):

- **Daily Contributions Log Table (#contribDaysTable):** Project / Task Contributed and Category columns render pure, clean, unhighlighted, uncolored text across Desktop and Mobile interfaces and Light and Dark themes (no green glowing pills, no colored borders, no background highlights), ensuring a distraction-free, professional corporate accounting log.
- **Day Status & Hours:** Clean, uncolored indicators without bright green or blue text.

5.2 Dynamic Multi-Year Month Selection Engine

To support long-range planning across multi-year engineering projects without artificial ceilings, the application incorporates a dynamic multi-year month engine that allows selecting any month and any future year while strictly preserving the November 2026 start boundary:

- **November 2026 Invariant (`APP_START_MONTH = '2026-11'`):** Earliest selectable boundary is locked to November 2026. Previous Month navigation arrow (`#prevMonthBtn`) prevents navigation prior to 11/2026. Date range inputs across Bulk Fill, Analytics, and Contributions modals enforce `finalStart >= 2026-11-01`.
- **Infinite Future Horizon:** Pre-populated in markup with 170+ options from 11/2026 to 12/2040. When `#nextMonthBtn` reaches the last option, `appendYearToMonthSelect(lastY + 1)` dynamically appends subsequent years on demand (2041, 2042, 2050+).
- **Dual-Mode Responsive Labels (`getMonthOptionLabel`):** Formats options dynamically based on device viewport. Desktop PC uses full month names with years (e.g. `11/2026 - November 2026 (Start)` , `08/2035 - August 2035`), while Mobile uses compact 3-letter month formatting (e.g. `11/2026 - Nov (Start)` , `08/2035 - Aug`).

- **Subsystem Synchronization:** `#auditMonthSelect`, `#contribMonthSelect`, `#projectMonthSelect`, and `#analyticsMonthFilterSelect` dynamically inherit all future months from `#monthSelect`.

5.3 Special Action Separation & Exclusive Top Placement

To preserve clean project accounting semantics and eliminate duplicate action items from the searchable catalog, special allocation actions (`DAY OFF`, `NO PROJECT`, `CLEAR/DELETE`) are isolated from the standard project catalog:

- **Exclusive Top Row Placement (`.quick-pill-row`):** Positioned as prominent, capital-letter, unhighlighted buttons at the top of each project selector surface (Cell Popover `#cellPopover`, PC Bulk Fill `#bulkBrowseSection`, and Mobile Logger `#mobileBrowserContent`).
- **Exclusion from Scrollable List (`.project-pill-list`):** Special action items are completely removed from the scrollable list container across all interfaces, reserving the catalog strictly for genuine projects, tenders, and training/support accounting entries.
- **Active State Feedback:** When an active cell or allocation holds `Day OFF`, `NO PROJECT`, or `__CLEAR__`, the corresponding top button dynamically displays `.is-selected-proj` selection styling.

6. Mobile Interface Architecture & Touch Physics

- **Dual Modes:** Mobile Editor mode (cards, agenda, 1-tap starred chips) vs. Read-Only Grid mode (horizontal table view).
- **Dual-Axis Touch Physics:** All table wrappers enforce `touch-action: pan-x pan-y !important` and `overflow-x: auto !important`, ensuring effortless left/right swipe navigation on phone displays.
- **Add Member Form Baseline Alignment:** `#addMemberBox .form-grid-3` enforces `align-items: flex-end` with `min-height: 28px` label containers so Full Name, Section, and Default Daily Hours inputs share an exact horizontal baseline.
- **Profile-Isolated Starred Projects:** Stored per-user in `localStorage` under `ixl_starred_{userId}`. 1-tap logging chips appear at the top of the mobile logger.

7. Role-Based Access Control & View-Only Defense

Role	Identification	Capabilities & Boundaries
View-Only	`?mode=view` URL query	Read-only inspection. All buttons, popovers, drawers, and sync requests are hard-blocked via CSS and JS guards.
Team Member	Chosen identity (e.g. Sarah KHALED)	Can edit only their own row. All other team member rows are read-only.
Master Admin	Peter MOUNIR (PIN-authenticated)	Full global rights: edit any row, 2D drag across team, add/deactivate members, move sections, adjust project baselines.

8. Automated Test Harness (12 Suites / 2,150+ Assertions)

Test Suite	Tests	Core Verifications
`test_all_users_integrity.js`	1,822	Row edit permission isolation across all 26 users; popovers, keyboard delete, audit events.
`test_suite.ps1`	92	Database schema, Cairo/Istanbul rules, timezone rollover, DOM elements, PIN hashing.
`test_starred_projects_integrity.js`	52	Profile isolation of starred favorites, 1-tap chips, star protection for special items.
`test_cell_click_hold_sequence.js`	47	380ms hold timer, 8px micro-jitter deadband, standing focus, arrow navigation.
`test_calculation_integrity.ps1`	39	Double-entry accounting balance, baseline hours arithmetic, 1h-12h boundary clamps.
`test_weekend_hours_assignment.js`	28	Cairo vs. Istanbul weekend range logic vs. single-day overtime assignments.
`test_pc_view_only_integrity.js`	27	PC read-only URL defense: DOM button removal, popover blocking, key blocking.
`test_mobile_view_only_integrity.js`	27	Mobile read-only URL defense: grid lock, form hiding, apply mutation rejection.
`test_mobile_project_browser.js`	25	Mobile logger agenda selection, integer hour stepping (1-12h), special project naming.
`test_mobile_list_scroll_alignment.js`	23	Mobile table minimum widths, 2-axis touch physics, Add Member form baseline alignment.
`test_pc_bulk_clear_integrity.js`	15	Bulk fill clear action normalization, star immunity, Projects List chooser.
`test_pc_welcome_screen_integrity.js`	11	PC and mobile welcome splash full-bleed screen fit, spinner placement, asset parity.
`test_dynamic_month_selection.js`	8	Dynamic month engine (Nov 2026 to 2040+), boundary clamps, responsive PC/mobile labels.
`test_quick_pill_separation.js`	6	Exclusive top quick-pill-row placement for DAY OFF, NO PROJECT, CLEAR/DELETE.

9. Modal Footer Clean Interface Architecture

To eliminate visual clutter and ensure a clean, modern aesthetic across both Desktop (PC) and Mobile interfaces:

- **Projects & WBS Management (`#projectsModal`):** The legacy helper message (*Double-click any project to enter a custom start/end date range...*) is completely removed from the modal footer. The modal footer cleanly displays only the right-aligned `Close` button.
- **Project Analytics / Monthly Data (`#projectMonthlyModal`):** The legacy helper message (*Double-click any project name in the projects list...*) is removed from the modal footer, maintaining full visual parity with `#analyticsModal` and `#projectsModal`.

10. Application Boot Sequence & Welcome Screen Resilience

The application employs a multi-tiered boot sequence to guarantee that the splash screen (`#mobileWelcomeOverlay`) never blocks entry or freezes:

- **Boot Order:** `initMobileWelcomeScreen()` binds click, touch, keydown, and 10000ms auto-dismiss handlers at step 1 before any layout or data requests execute.
- **Inline Markup Fallback:** The overlay element in `index.html` specifies an inline `onclick` handler that triggers CSS fade-out and unmounts the overlay even in the event of script loading interruptions.
- **Safety Fallback Timer:** A lightweight inline script guarantees dismissal after 6000ms if no interaction occurs.

11. Developer Guidelines & Ongoing Maintenance Contract

1. **Mandatory Document Update:** Whenever you add, modify, or delete any feature, data model, or style rule, you MUST immediately update both `TECHNICAL_ARCHITECTURE.md` and `TECHNICAL_ARCHITECTURE.html`.
2. **Strict Zero-Emoji Policy:** Never use Unicode emojis in code, commits, or documentation.
3. **Preserve Double-Entry Math:** Always run `test_calculation_integrity.ps1` after altering hours calculation logic.
4. **CRLF Line Endings:** Preserve Windows CRLF (`\r\n`) line endings across repository files.
5. **Timezone Safety:** Always construct dates via `parseLocalDate(dateStr)`, never using raw UTC midnight strings.
6. **Cache Busting:** When updating `styles.css` or `app.js`, bump the cache query parameter in `index.html` (e.g. `?v=10.0`).